ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN (UIT)

BÁO CÁO BÀI TẬP THỰC HÀNH

ỨNG DỤNG TỪ ĐIỂN TIẾNG ANH

Cấu trúc dữ liệu Radix-Trie

Môn học: Cấu trúc dữ liệu và giải thuật nâng cao - CS523.Q21
Sinh viên thực hiện: Trần Minh Thiện
Mã số sinh viên: 24521668
Mã nguồn Github: Link Github

Năm học 2025 - 2026

1 Tổng quan Ứng dụng và Cấu trúc dữ liệu

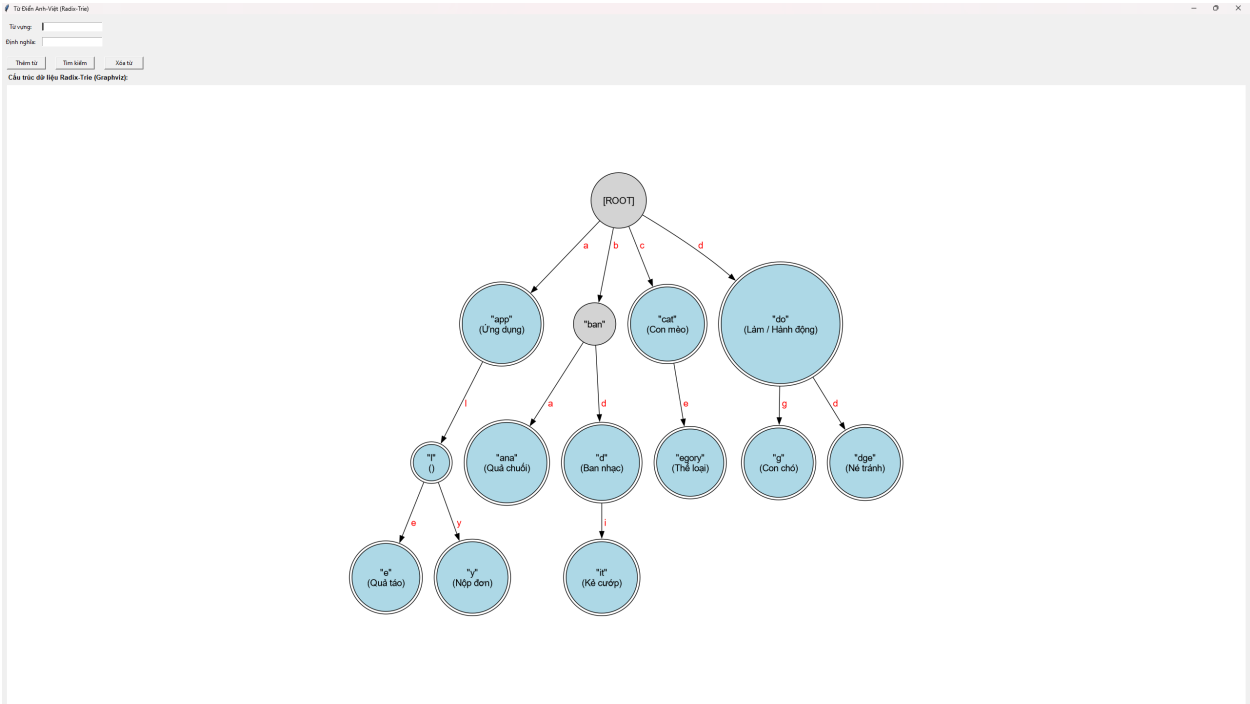
Ứng dụng Từ điển Tiếng Anh được phát triển với giao diện trực quan, cho phép người dùng thực hiện các thao tác cơ bản: Thêm, Xóa, và Tìm kiếm mục từ.

Điểm nhấn kỹ thuật của ứng dụng là việc sử dụng cấu trúc dữ liệu **Radix-Trie (Patricia Tree)** thay vì Hash Table hay Standard Trie thông thường.

- **Lý do lựa chọn:** Radix-Trie tối ưu hóa không gian bộ nhớ bằng cách gộp các node có chung tiền tố không phân nhánh lại với nhau (mỗi đỉnh chứa một chuỗi ký tự thay vì chỉ 1 ký tự).
- **Độ phức tạp tra cứu:** Đạt mức $O(k)$ với k là chiều dài của từ, cực kỳ tối ưu cho các hệ thống từ điển yêu cầu tốc độ phản hồi tức thời.

2 Giao diện chương trình

Giao diện được xây dựng bằng thư viện Tkinter kết hợp công cụ `Graphviz` để trực quan hóa sơ đồ cây ngay thời gian thực. Bất kỳ sự thay đổi nào về mặt dữ liệu đều được cập nhật và vẽ lại lập tức lên màn hình.

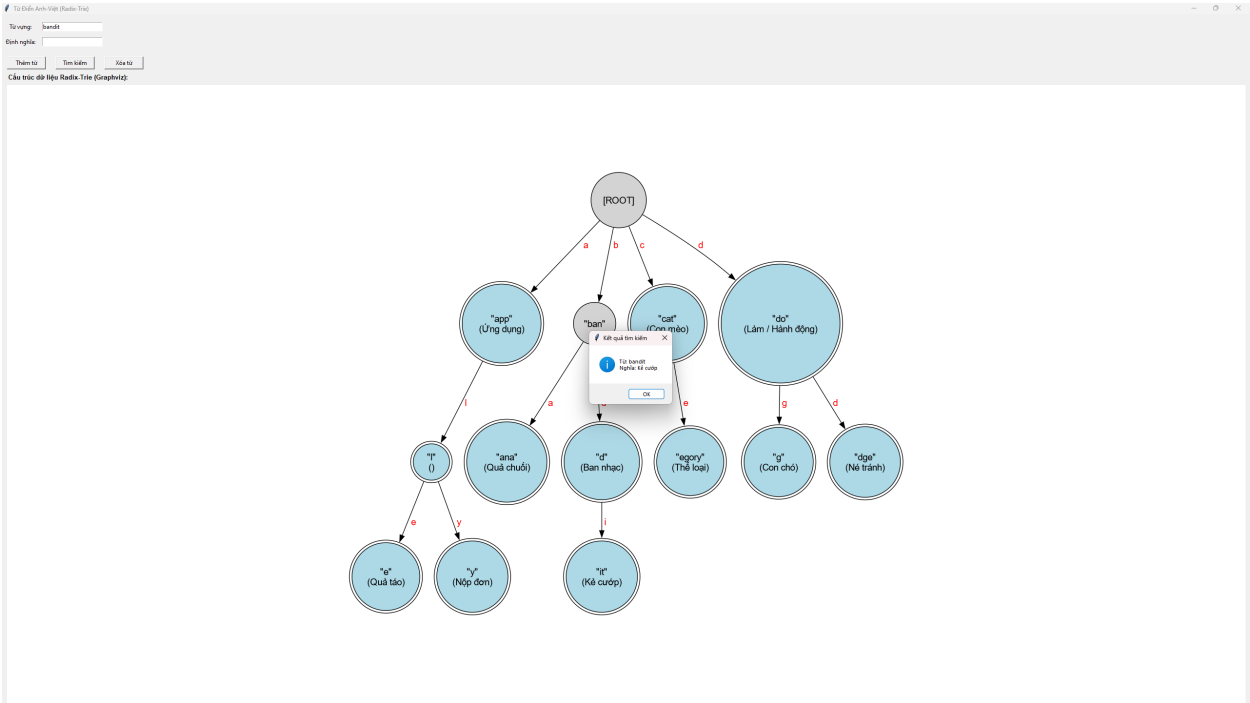


Hình 1: Giao diện tổng quan của ứng dụng và cấu trúc Radix-Trie khởi tạo.

3 Mô phỏng các thao tác và sự thay đổi dữ liệu

3.1 Thao tác Tra cứu (Search)

Thuật toán duyệt cây theo tiền tố. Nếu chuỗi truy vấn khớp hoàn toàn với đường đi trên nhánh và dừng lại tại một node có cờ `isEndOfWord = True`, ứng dụng sẽ trả về định nghĩa.



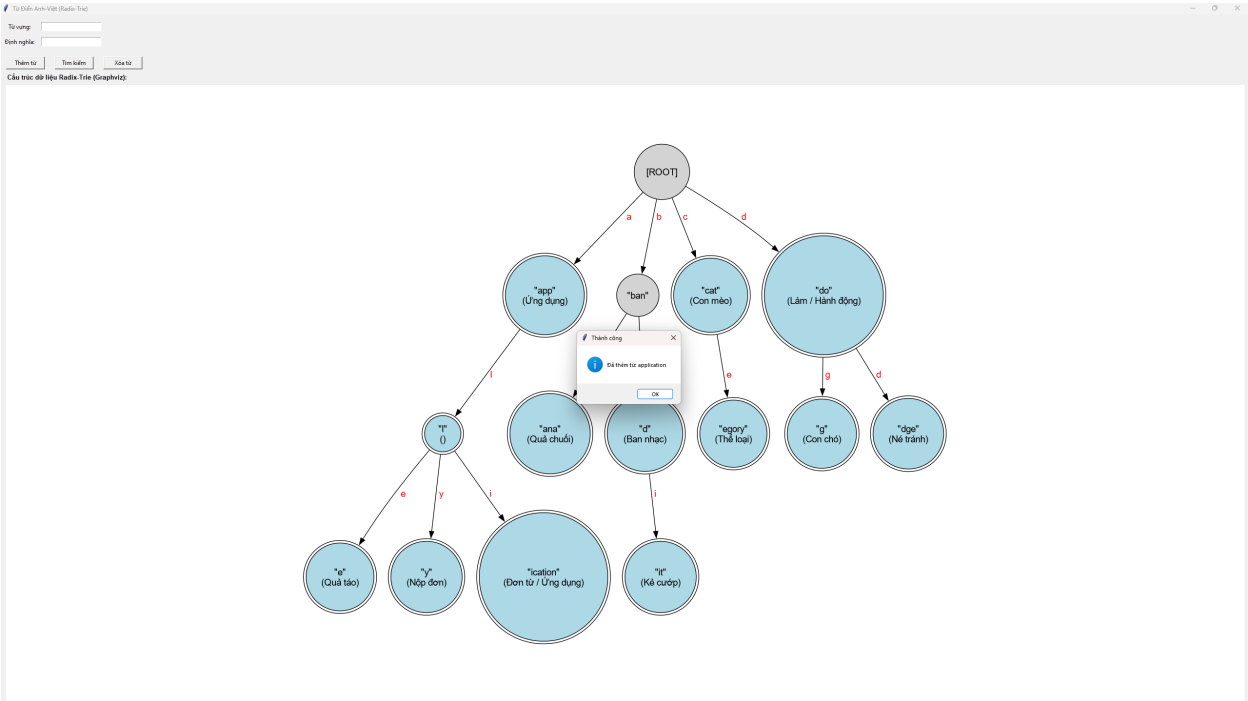
Hình 2: Thông báo kết quả khi tìm kiếm thành công từ "bandit".

3.2 Thao tác Thêm (Insert) & Cơ chế Chia tách (Split)

Đây là thao tác phức tạp nhất, thể hiện rõ tính chất của Radix-Trie. Khi thêm một từ mới có một phần tiền tố trùng với nhánh đã có, thuật toán phải tiến hành **Cắt nhánh (Split)**.

Kịch bản minh họa:

- Đang có sẵn các từ nhánh "app"gồm: *app, apple, apply*. Nhánh `"appl"` đang rẽ ra 2 nhánh `"e"` (apple) và `"y"` (apply).
- Nhập từ mới **"application"** và nhấn Thêm.
- **Sự thay đổi dữ liệu:** Từ "application"có chung tiền tố "appli"với "apply". Do đó, nhánh chữ `"y"` (apply) bị gãy. Cây tự động sinh ra một node trung gian mới là `"i"`, sau đó rẽ thành `"y"` (apply) và `"cation"` (application).



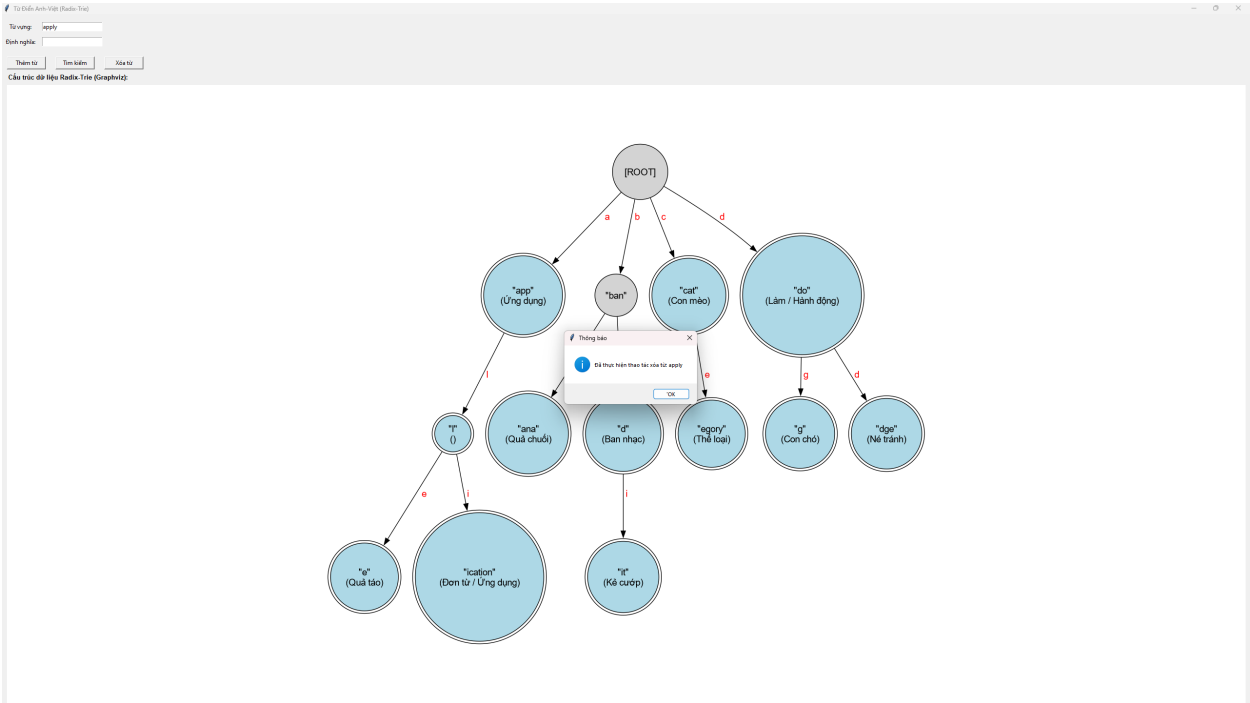
Hình 3: Dữ liệu được tái cấu trúc (Split) khi chèn thêm từ "application". Nhánh "app"→ "l"→ "i"xuất hiện.

3.3 Thao tác Xóa (Delete) & Cơ chế Gộp nhánh (Merge)

Trong Radix-Trie, việc xóa không chỉ là gỡ bỏ định nghĩa, mà phải tối ưu lại bộ nhớ. Nếu sau khi xóa, một node trung gian mất đi trạng thái kết thúc từ và chỉ còn lại duy nhất một node con, nó bắt buộc phải **Gộp (Merge)** với node con đó.

Kịch bản minh họa:

- Xóa từ **"apply"** khỏi cây vừa tạo.
- **Sự thay đổi dữ liệu:** Node `"y"` biến mất. Lúc này, node trung gian `"i"` chỉ còn lại một nhánh con duy nhất là `"cation"`. Để bảo toàn tính chất tối ưu, thuật toán tự động gộp `"i"` và `"cation"` lại thành một node duy nhất là `"ication"`.



Hình 4: Tính năng tự động gộp nhánh (Merge Compaction) sau khi xóa. Nhánh "i"hợp nhất thành "ication".

4 Tổng kết

Ứng dụng đã mô phỏng thành công cách hoạt động của Radix-Trie, đảm bảo tính toàn vẹn của dữ liệu trong quá trình Thêm, Xóa, Tìm kiếm. Việc tích hợp Graphviz giúp sinh viên và người dùng dễ dàng theo dõi sự biến đổi cấu trúc bộ nhớ bên dưới (under-the-hood) một cách trực quan, làm rõ được sự ưu việt của thuật toán tối ưu tiền tố.